

TP - Bataille en POO

Constructeur

1. Dans le dossier **NSI**, créer un nouveau dossier **POO**.
2. Créer un nouveau fichier Python nommé *carte.py* (<http://carte.py>). Copier la classe Carte et son constructeur.
3. **Dans le programme principal** (en dehors de la classe), créer une nouvelle carte de votre choix.
Afficher la carte grâce à la fonction `print`. Quel est le résultat ?
4. A votre avis, comment peut-on afficher la couleur de la carte seulement ?
5. Afficher ses autres attributs.
6. Modifier la valeur de la carte puis l'afficher.
7. Modifier la couleur de la carte pour qu'elle ait la valeur `vert` puis affichez la. Quel problème cela pose ? Quelles sont les solutions possibles ?

Accesseurs et mutateurs

Les attributs sont maintenant considérés comme **privé**. On ne peut plus y accéder ou les modifier directement dans le programme principal.

1. Ajouter les méthodes `get_valeur()` et `set_valeur()` vues en cours après le constructeur (toujours dans la classe).
2. Créer une nouvelle carte avec les données de votre choix.
Modifier sa valeur en appelant la méthode `set_mail()` que vous venez d'écrire, puis affichez-la (avec `get_valeur()`) afin de vérifier que la modification s'est bien effectuée.
3. Modifier la classe en ajoutant les accesseurs et mutateurs pour la couleur. Tester les nouvelles méthodes.

Afficher un objet et méthodes spéciales

Pour notre jeu de carte, on veut pouvoir afficher proprement une carte, **avec toutes sa valeur et sa couleur**. Sous la forme : *10 de trèfle* par exemple.

1. On a vu que `print(objet)` n'affiche pas les valeurs des attributs de l'objet. Qu'est-ce que cela affiche ?

2. Ajouter une **méthode** dans la classe qui retourne une **chaîne de caractères**, correspondant à **l'affichage des données d'une carte**. On appellera la méthode `afficher()`. Quels seront ses paramètres ?

3. Créer une nouvelle carte avec les données de votre choix. Puis appeler la méthode `afficher()` sur cette carte. Que se passe-t-il ?

4. Utiliser l'instruction `print(objet.afficher())` pour afficher toutes les cartes créées.

On peut voir que la syntaxe est un peu lourde. Heureusement, il existe des **méthodes spéciales** qui portent des noms particuliers et ont des **rôles spécifiques**. Comme pour le constructeur vu précédemment.

5. Renommer la méthode `afficher` en `__str__`.

6. Afficher les cartes avec l'instruction `print(objet)`. Que se passe-t-il ?

Le fameux `print` utilisé si souvent dans nos programme est en réalité une méthode codée différemment selon le type de l'objet à afficher.

Il existe de nombreuses méthodes de ce genre, nous en utiliserons selon nos besoins. `__init__` et `__str__(self)` sont les plus utilisées en général.

► En voici quelques unes parmi les plus importantes :

7. Ajouter la méthode `__eq__(self, carte2)` à la classe `Carte`. Cette méthode doit comparer l'objet `self` à l'objet `carte2`. Elle renvoie **True** si la valeur de `self` est égale à la valeur de `carte2`, **False** sinon.

8. Ajouter la méthode `__gt__(self, carte2)` à la classe `Carte`. Cette méthode doit comparer l'objet `self` à l'objet `carte2`. Elle renvoie **True** si la valeur de `self` est supérieure à la valeur de `carte2`, **False** sinon.

9. Tester les nouvelles méthodes grâce au programme ci-dessous :

```
1  carte1 = Carte(3, 'pique')
2  carte2 = Carte(3, 'carreau')
3  carte3 = Carte(7, 'trèfle')
4
5  print(carte1 == carte2) # Affiche True
6  print(carte1 > carte3) # Affiche False
```

Gérer plusieurs classes : le paquet de carte

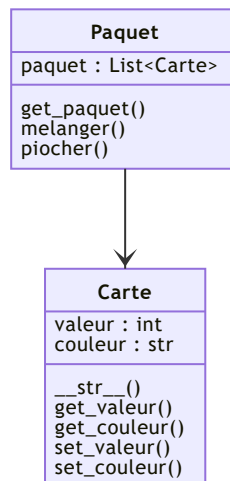
Maintenant que notre classe Carte est fonctionnelle, on peut passer la création d'un paquet de carte.

On va donc créer une nouvelle classe représentant une liste de cartes, que l'on appellera Paquet. Cette classe sera donc liée à la classe Carte que nous venons de faire.

Il est important de bien s'organiser quand on gère plusieurs classes. Chaque classe doit contenir toutes les informations et actions possibles des objets qu'elle crée.

On veut pouvoir :

- Créer un paquet de 52 cartes
- Mélanger le paquet
- Piocher une carte du paquet



On prendra pour habitude de créer **un fichier par classe**, ainsi qu'un fichier supplémentaire pour le **programme principal**.

1. Dans votre dossier **POO**, créer un nouveau fichier nommé *Paquet.py* (<http://Paquet.py>).

2. Maintenant à vous **d'implémenter la classe** **Paquet** . On se posera les questions suivantes :

- Comment utiliser des méthodes de la classe `Carte` ?
- Quels sont les attributs d'un paquet de Cartes ?
- Comment voulez-vous instancier un nouveau paquet ? (initialisation des attributs)
- Quelles vont être ses différentes méthodes et leur impact sur les attributs ?
- Vous pouvez également la personnaliser comme vous le souhaitez en ajoutant des fonctionnalités ou des petites particularités.

- 3.** Commencer par le constructeur. Comment créer les 13 cartes de chaque couleur (trèfle, pique, carreau et coeur) et les mettre dans une liste ?
- 4.** La méthode `get_paquet()` renvoie seulement la liste de cartes.
- 5.** La méthode `melanger()` modifie l'ordre des cartes du paquet mais ne renvoie rien. Utiliser la méthode `list.shuffle()` .
- 6.** La méthode `piocher()` renvoie la première carte du paquet et la supprime.
- 7.** Créez un nouveau fichier nommé `main.py` pour votre programme principal, puis à vous de construire un paquet de cartes et d'agir dessus en testant toutes les méthodes.